AgFarm " Redux Toolkit (RTK) Complete Guide

Architecture, Slices, State Persistence & Frontend UI Integration

1. Frontend UI Diagnosis & Resolved Issues

Why was the frontend UI not showing or appearing inactive?

1. Vite Network Binding: By default, Vite binds to [::1] (IPv6 loopback). On Windows, browsers attempting to access http://localhost:5173 or 127.0.0.1 can fail if IPv4/IPv6 resolution is misconfigured.
   Fix Applied: Configured server: { host: true, port: 5173 } in vite.config.ts to bind to 0.0.0.0 & ::.
2. Ephemeral Redux Auth State: authSlice was resetting user and accessToken to null on refresh.
   Fix Applied: Implemented automatic localStorage synchronization in authSlice (initialState & reducers).
3. Missing Bearer Token: Axios instance in src/lib/api.ts was not injecting the Authorization header.
   Fix Applied: Added an Axios request interceptor that automatically attaches Bearer <token>.
4. Missing Typed Hooks & Barrel Export: Official RTK recommends useAppDispatch & useAppSelector.
   Fix Applied: Created src/store/hooks.ts and src/store/index.ts barrel exports.
5. Missing Route Fallback: Any unmatched route rendered blank. Added <Route path="*" element={<Navigate to="/" />} />.

---

2. Redux Toolkit (RTK) Core Architecture

Redux Toolkit is the official, opinionated, batteries-included toolset for efficient Redux development.
It solves the 3 major classic Redux headaches: boilerplate setup, complex store configs, and immutability bugs.

Key Building Blocks of Redux Toolkit:

- " configureStore(): Simplifies store setup with Redux Thunk and Redux DevTools enabled by default.
- " createSlice(): Generates action creators, action types, and reducers in one unified declarative block.
- " Built-in Immer: Lets you write "mutative" code (state.user = action.payload) that safely creates immutable copies.
- " createAsyncThunk(): Handles asynchronous lifecycles (pending, fulfilled, rejected) cleanly.

3. Implementing Slices with createSlice

Below is the exact production implementation of authSlice.ts with safe state persistence:

```
import authSlice from "./slices/authSlice"
import { createSlice, type PayloadAction } from '@reduxjs/toolkit'









import uiSlice from './slices/uiSlice'
```

4. Configuring the Central Store (store.ts)

```
import { configureStore } from '@reduxjs/toolkit'
```

5. Typed React-Redux Hooks (hooks.ts)

Instead of importing untyped useDispatch/useSelector into components, use pre-typed hooks:

```
import { useDispatch, useSelector } from 'react-redux'
```

6. How to Use in React Components (e.g., Navbar / Login)

Read state using useAppSelector and update state using useAppDispatch:

```
import { useAppDispatch, useAppSelector, setCredentials, clearCredentials } from '@/store'
```

7. How to View the Frontend UI

1. Open your browser and navigate to: http://localhost:5173 or http://127.0.0.1:5173
2. The dev server is already running! (Vite now responds on both IPv4 and IPv6).
3. Test the navbar: Click "Sign In" or "Register" to test Redux authentication in real-time!